

EDUGENIE: AI LEARNING ASSISTANT

II – BSc Artificial Intelligence and Machine Learning

TEAM LEADER:

MAHALAKSHMI – mahalakshmiifs@gmail.com

TEAM MEMBERS:

T. THIRISHA – thirishabanumathi@gmail.com

K. KOWSIKA – kowsikaanandhan@gmail.com

K. YUVASREE – yuvasree599@gmail.com

PROJECT DOCUMENTATION

EduGenie

AI Learning Assistant powered by Google Gemini

*A FastAPI + Google Gemini web application for question answering,
concept explanation, quiz generation, summarization and learning-path guidance*

1. Introduction

EduGenie is a lightweight, browser-based AI learning assistant that brings together several everyday study tasks — asking questions, understanding new concepts, generating quizzes, summarizing notes, and planning a learning path — into a single web page. The application is built on a FastAPI backend and a simple HTML/CSS frontend rendered through Jinja2 templates, and it draws its intelligence from the Google Gemini API.

Rather than offering a separate form for every feature, EduGenie presents one unified “AI Learning Assistant” screen. The learner picks the task they want from a “Select Task” dropdown (for example, Ask a Question or Recommend Learning Path), types their question or content into a single text box, and clicks Generate. The request is sent to the FastAPI backend, processed by the relevant Python module, passed on to the Gemini model, and the AI-generated response is rendered back on the same page under a “Result” heading — without a page reload.

The goal of the project is to give students a fast, low-friction study companion that runs locally with minimal setup, while still being powered by a capable large language model in the cloud.

2. Problem Statement

Students preparing for exams or trying to learn a new subject typically have to juggle several disconnected tools: a search engine for quick answers, a separate app or tutor for explanations, another tool to generate practice questions, and yet another to summarize long reading material or plan out a study schedule. Switching between these tools breaks concentration and slows down learning.

There is a need for a single, lightweight application that a student can open locally, type a question or topic into, and immediately get a useful, well-structured response — whether that response is a direct answer, a simplified explanation, a quiz, a summary, or a full learning roadmap.

3. Objectives

- Provide instant, AI-generated answers to student questions through a simple text prompt.
- Break down complex or unfamiliar topics into short, easy-to-understand explanations.
- Generate multiple-choice quizzes on demand to help learners test their understanding.
- Summarize long educational passages into concise, revision-friendly notes.
- Recommend a structured, beginner-to-advanced learning path for any subject the learner names.
- Keep the application lightweight and easy to run locally with a single command.

4. Features

- **Ask a Question** — accepts any question typed by the user and returns a structured response containing a Direct Answer, a Simple Explanation, and, where useful, an Example.
- **Concept Explanation** — explains a topic or term in simple, beginner-friendly language.

- **Quiz Generation** – generates a multiple-choice quiz on a chosen topic. The assistant can be instructed (through the prompt) to ask a fixed number of questions with four options each, present them one at a time, mark each answer as correct or wrong with a brief explanation, and show a final score and percentage.
- **Content Summarization** – condenses long paragraphs or study material into a short Direct Answer, a Simple Explanation, and an illustrative Example, ideal for quick revision.
- **Learning Path Recommendation** – produces a complete, stage-wise learning roadmap for a subject — covering the learning goal, beginner-to-advanced stages, topics for each stage, practice exercises, a small project, and prerequisites for moving to the next stage.

5. Modules

EduGenie's backend is organized into small, single-purpose Python modules, each responsible for one feature. This keeps the AI-prompting logic for every task isolated and easy to maintain.

Folder Structure

File / Folder	Purpose
main.py	FastAPI application; defines all REST endpoints and routes each request to the correct module
gemini_client.py	Shared client that connects to the Google Gemini API and sends prompts on behalf of the other modules
qna.py	Handles the Ask a Question feature
explanation_module.py	Handles the Concept Explanation feature
quiz_module.py	Handles Quiz Generation
summary_module.py	Handles Content Summarization
learning_path.py	Handles Learning Path Recommendation
templates/index.html	Jinja2 HTML template for the single-page frontend
static/style.css	Styling for the frontend page
tests/	Automated test cases for the backend modules
.env	Stores the Gemini API key and other configuration values

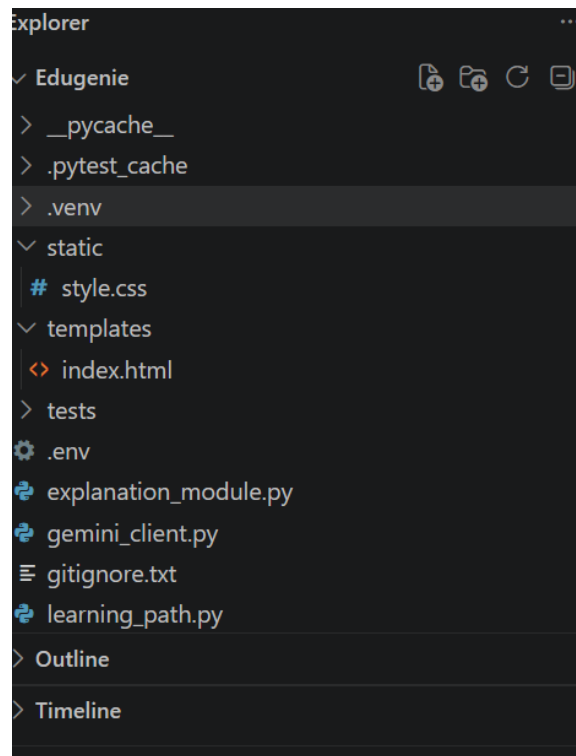


Fig. 5.1 – EduGenie Project Folder Structure (VS Code Explorer)

Module Descriptions

gemini_client.py – wraps the Google Gemini API so that every other module can send a prompt and get back generated text through one shared function, instead of repeating API setup code in each module.

qna.py – takes the learner's question, forwards it to Gemini, and returns a structured answer that the frontend displays under Direct Answer, Simple Explanation and Example headings.

explanation_module.py – builds a prompt asking the model to explain a given topic in simple terms and returns the simplified explanation.

quiz_module.py – builds the quiz-generation prompt (number of questions, number of options, one-question-at-a-time flow, scoring at the end) and returns the model's quiz output for display.

summary_module.py – sends the passage or topic to be summarized to Gemini and returns a condensed, easy-to-read summary.

learning_path.py – builds a detailed prompt asking Gemini for a stage-wise learning roadmap (goal, topics, exercises, a project and prerequisites for every stage) and returns the generated plan.

main.py – wires everything together as a FastAPI app: it exposes endpoints (including a summarize endpoint and a /learn/recommendations endpoint, as seen in the code), receives requests from the frontend, calls the matching module, and returns the AI-generated result as JSON.

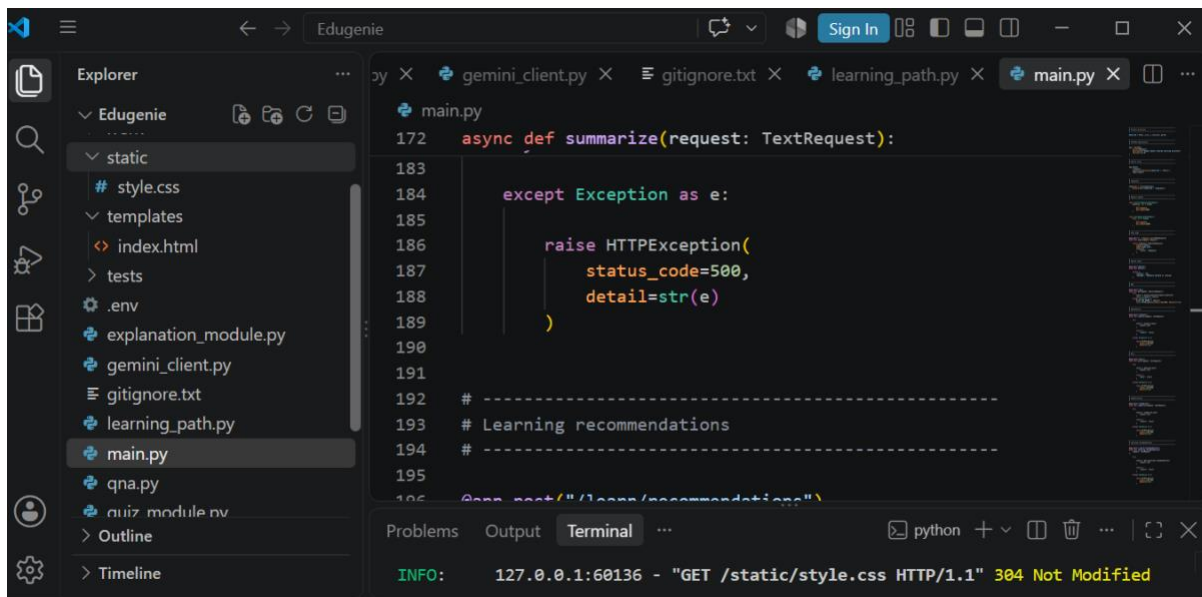


Fig. 5.2 – main.py: FastAPI Backend Endpoints (Summarize and Learning Recommendations)

Frontend Template

The interface is a single Jinja2 template, templates/index.html, styled with static/style.css. The template defines the page title (“EduGenie | AI Learning Assistant”), links the stylesheet using Jinja2's url_for helper, and lays out the task dropdown, input box, Generate button and Result panel seen on the running application.

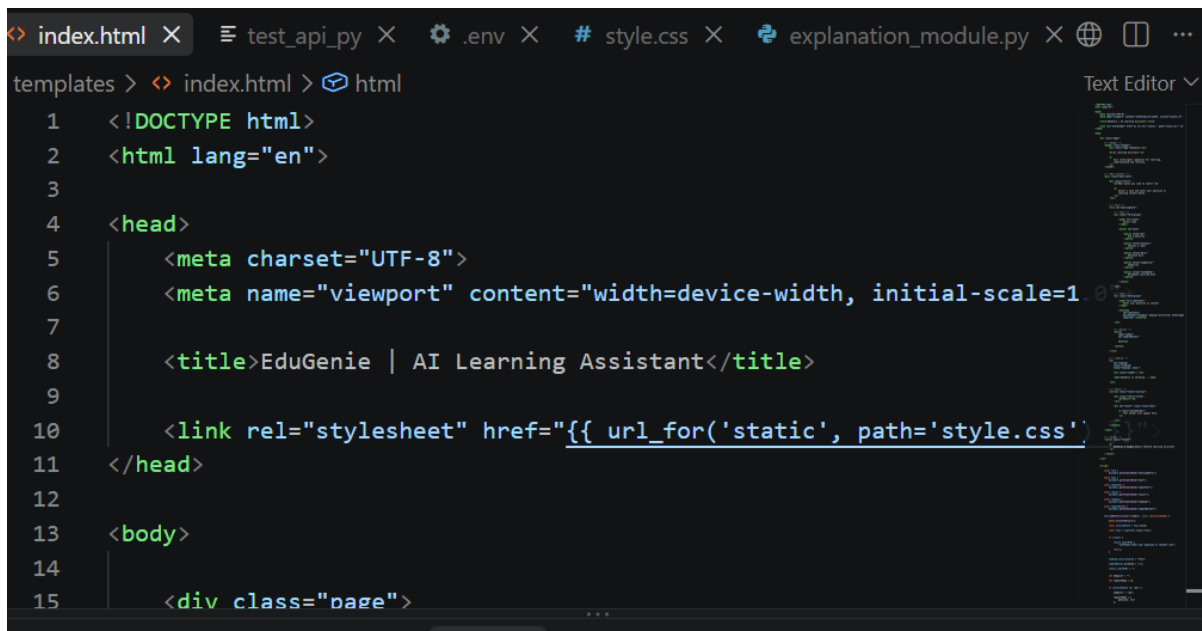
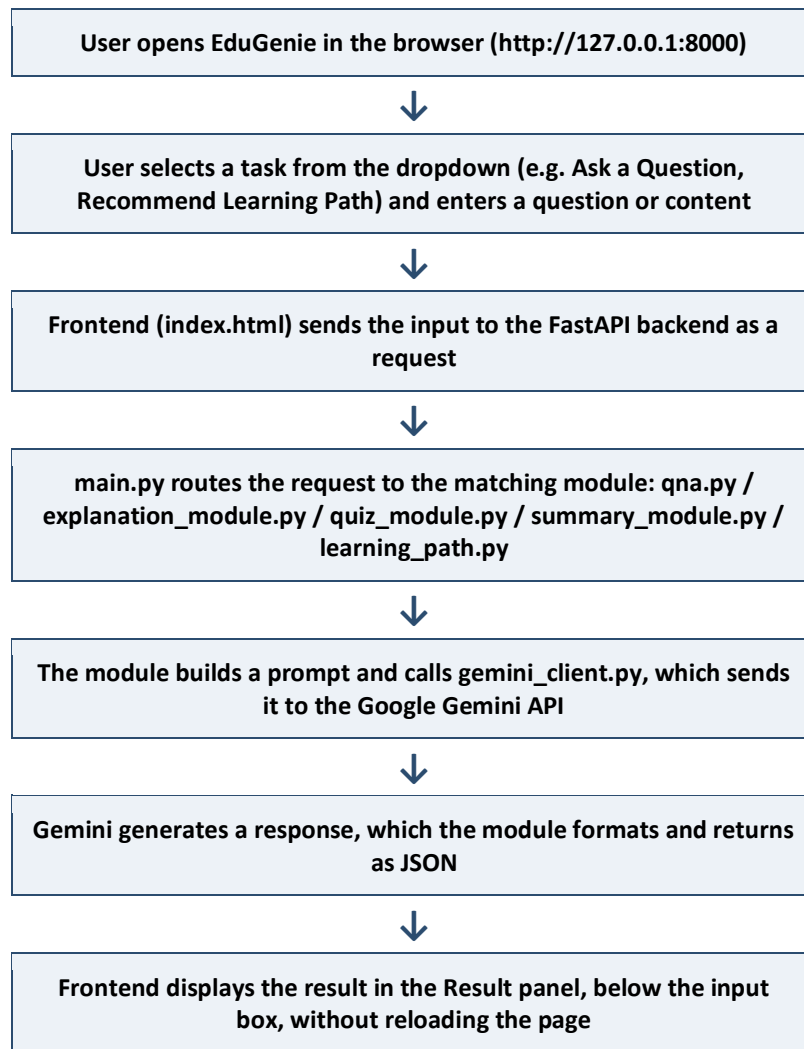


Fig. 5.3 – templates/index.html: Frontend Page Head and Stylesheet Link

6. System Workflow

The application follows a simple request/response flow between the browser, the FastAPI backend and the Gemini API:



7. Technologies Used

- **Python 3.10+** – core programming language for the backend.
- **FastAPI** – backend web framework used to define REST endpoints and handle requests.
- **Uvicorn** – ASGI server used to run the FastAPI application (started with the --reload flag during development, as seen in the terminal).
- **Jinja2** – HTML templating engine used to render templates/index.html and link static assets.
- **Google Gemini API** – generative AI model that powers question answering, explanations, quiz generation, summarization and learning-path recommendations.
- **HTML5 & CSS3** – used for the frontend page structure (templates/index.html) and styling (static/style.css).
- **.env configuration** – used to store the Gemini API key and other environment-specific settings outside the source code.
- **Visual Studio Code** – primary development environment used to write and debug the project.

- **pytest (tests/)** – used for writing and running automated tests for the backend modules.

8. UI / Screen Description

The application opens on a single page titled “AI Learning Assistant” with the tagline “Your intelligent companion for learning, understanding and revision.” It contains a Select Task dropdown, a text area for the question or content, a Generate button, and a Result panel where the AI's response appears.

a. Ask a Question

With Ask a Question selected in the dropdown, the user types a plain question — for example, “what is java” — and clicks Generate. The Result panel returns a structured answer broken into a Direct Answer, a Simple Explanation, and further detail, giving the learner both a quick fact and a deeper, beginner-friendly explanation of the same concept.

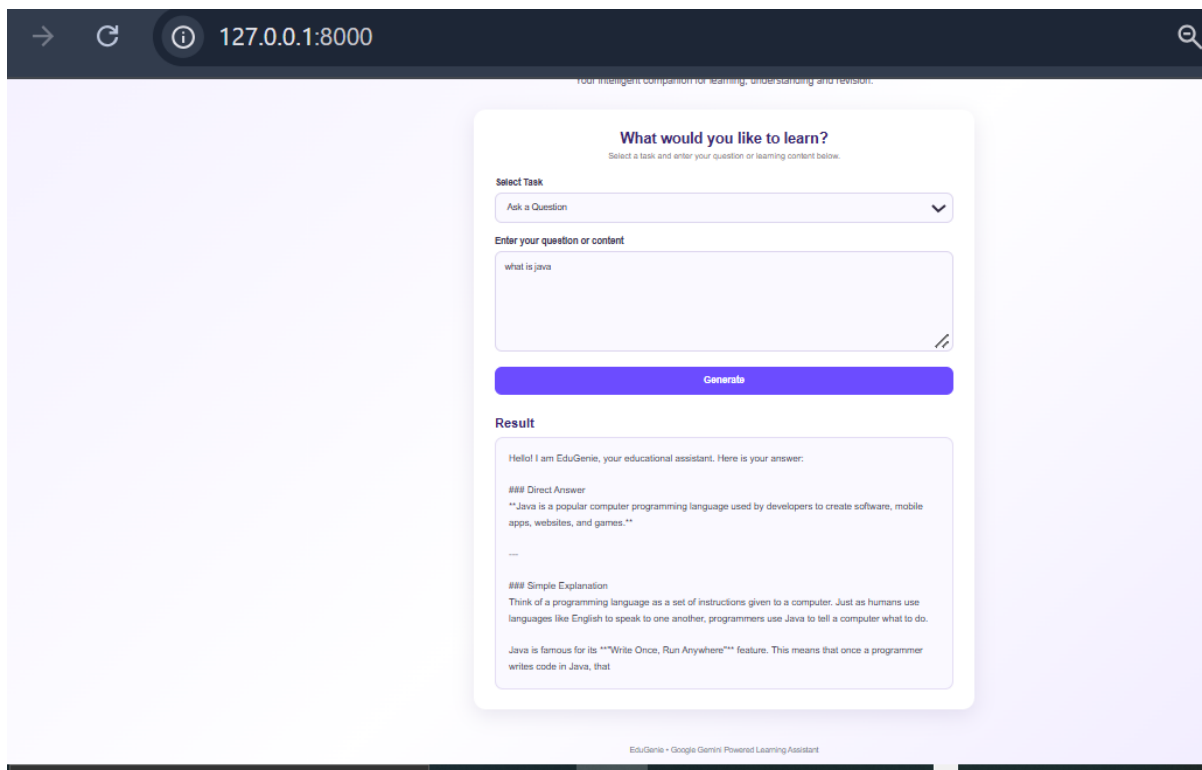


Fig. 8.1 – Ask a Question: Structured AI-Generated Answer

b. Content Summarization

The same Ask a Question box can also be used to request a summary, as shown below with the prompt “summarize what is python.” The Result panel returns a Direct Answer, a Simple Explanation, and a short code Example, condensing the topic into an easy, revision-ready format.

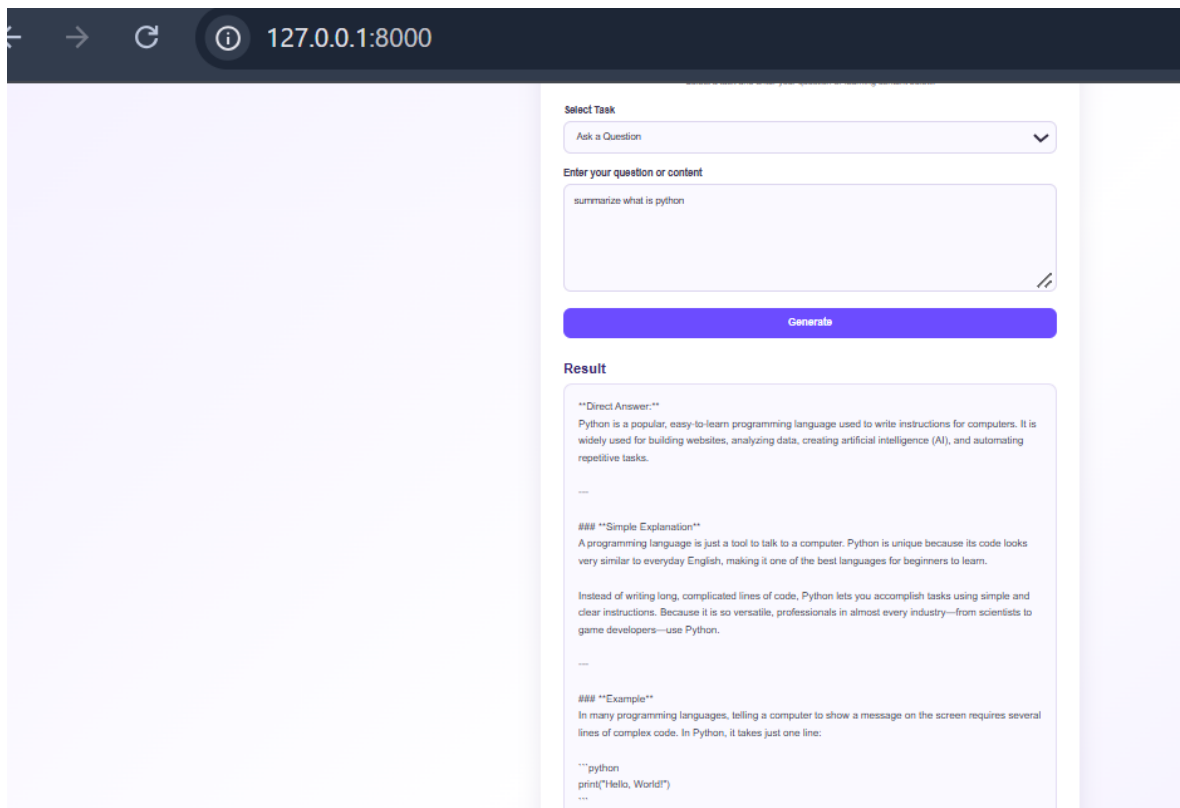


Fig. 8.2 – Summarization Output for a Requested Topic

c. Quiz Generation

For quizzes, the learner describes the quiz they want in the input box — for instance, a 10-question multiple-choice quiz on Python Basics, with four options per question, one question asked at a time, feedback after every answer, and a final score and percentage at the end. EduGenie responds by introducing itself and presenting the first question with four lettered options, ready for the learner to answer.

127.0.0.1:8000

AI Learning Assistant

Your intelligent companion for learning, understanding and revision.

What would you like to learn?

Select a task and enter your question or learning content below.

Select Task

Ask a Question

Enter your question or content

Create a 10-question multiple-choice quiz on Python Basics.
Give 4 options for each question.
Ask one question at a time and wait for my answer.
After I answer, tell me whether it is correct or wrong and explain the answer briefly.
At the end, show my total score and percentage.

Generate

Result

Hello! I'm "EduGenie", your AI educational assistant. Welcome to your 10-question "Python Basics" quiz!

We will go step-by-step, one question at a time. Here is your first question:

"Question 1 of 10"

Which function is used to display text or output on the screen in Python?

A) 'echo()'
B) 'display()'
C) 'print()'
D) 'output()'

Fig. 8.3 – Quiz Generation: Interactive Multiple-Choice Quiz

d. Learning Path Recommendation

With Recommend Learning Path selected from the dropdown, the learner describes what they want the plan to include — topics, objectives, order, exercises, a project, a quiz, and prerequisites for each stage — and names a subject (Python programming, in the example shown). EduGenie returns a complete roadmap starting with the overall Learning Goal, followed by a Beginner Level section listing topics to learn (such as Python basics, control flow and functions) along with practice exercises for each topic.

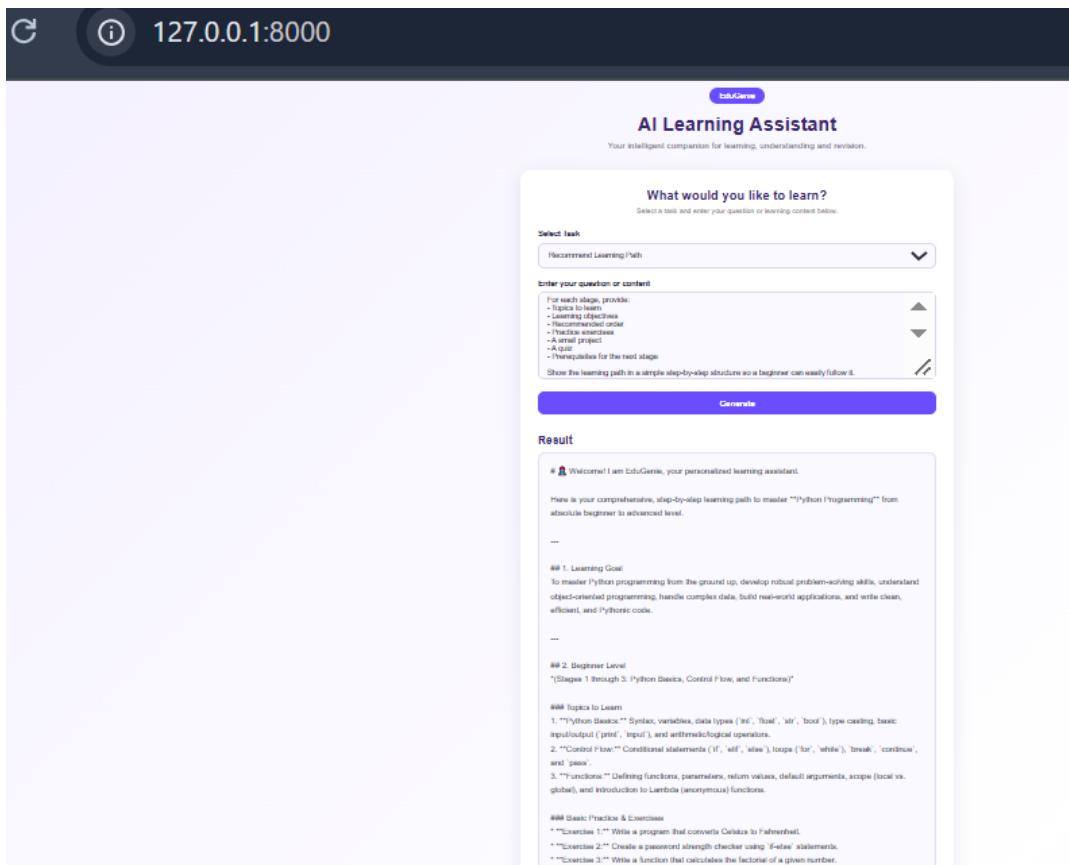


Fig. 8.4 – Recommend Learning Path: Stage-wise Learning Roadmap

9. Project Implementation

9.1 Setting Up and Running the Application

The backend is started locally using Uvicorn from a Python virtual environment. The terminal below shows the application being launched on Windows with the command `.venv\Scripts\python.exe -m uvicorn main:app --reload`, after which Uvicorn starts watching the project directory for changes, launches the reloader and server processes, and waits for the application to finish starting up.

```
S C:\Users\dell\Desktop\EduGenie> .venv\Scripts\python.exe -m uvicorn
reload
INFO:      Will watch for changes in these directories: ['C:\\Users\\
\\EduGenie']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to
INFO:      Started reloader process [3032] using WatchFiles
INFO:      Started server process [4340]
INFO:      Waiting for application startup.
```

Fig. 9.1 – Starting the FastAPI Server with Uvicorn

Once the server is running, the application is accessed locally by navigating to `http://127.0.0.1:8000` in a web browser, which loads the AI Learning Assistant page shown throughout Section 8.

9.2 Backend Implementation

The FastAPI backend in `main.py` defines the application's endpoints and wires each one to its corresponding module. The code excerpt in Fig. 5.2 shows the tail end of the `summarize` endpoint, with error handling that raises an `HTTPException` on failure, followed by the start of the `/learn/recommendations` endpoint used for the Learning Path feature. Each endpoint accepts the request from the frontend, calls the matching module (`qna`, `explanation`, `quiz`, `summary`, or `learning path`), and returns the Gemini-generated result as a JSON response.

9.3 Frontend Implementation

The frontend is a single HTML page rendered through Jinja2 (`templates/index.html`), styled by `static/style.css` and served by FastAPI's static file support. As shown in Fig. 5.3, the page declares a UTF-8 charset and a responsive viewport, sets the title to “EduGenie | AI Learning Assistant,” and links the stylesheet using `{{ url_for('static', path='style.css') }}`. The visible page — the Select Task dropdown, input box, Generate button and Result panel — is built inside the body of this template and populated dynamically with the response returned from the backend.

9.4 Functional Testing

The application was tested by exercising each task from the dropdown end-to-end through the browser:

- Asking a general knowledge question and verifying a structured answer is returned
- Requesting an explanation and a summary through the same input box
- Generating a multiple-choice quiz and confirming questions are presented one at a time with four options
- Requesting a learning path for a subject and confirming a full, stage-wise roadmap is returned
- Confirming the backend starts correctly with Uvicorn and serves the page at `http://127.0.0.1:8000`

10. Future Enhancements

- **Dedicated Quiz Interface** – a purpose-built quiz screen with clickable options, live scoring and a results summary, instead of driving the quiz through the general question box.
- **User Accounts & Progress Tracking** – saving each learner's quiz scores, summaries and learning-path progress across sessions.
- **Voice Interaction** – adding speech-to-text input and text-to-speech output for hands-free learning.
- **Multilingual Support** – allowing questions, explanations and quizzes to be generated in languages other than English.
- **Export Options** – letting users download a generated summary, quiz, or learning path as a PDF or note file.
- **Mobile-Friendly Interface** – adapting the layout for phones and tablets, or building a companion mobile app.

- **Analytics Dashboard** – visual charts showing quiz performance and learning-path completion over time.

11. Conclusion

EduGenie brings five common study tasks — asking questions, understanding concepts, generating quizzes, summarizing content, and planning a learning path — into one lightweight FastAPI application powered by the Google Gemini API. By routing every request through a single, consistent task-selection interface and a set of small, focused backend modules, the project keeps the codebase easy to understand while still delivering rich, AI-generated study material.

The working application, verified through local testing with Uvicorn and demonstrated across all five task types, shows that a small team (or individual developer) can build a genuinely useful AI study tool without a heavy infrastructure footprint. With the module-based structure already in place, EduGenie is well positioned for the future enhancements outlined above, from a dedicated quiz experience to multilingual and voice-based learning.